

GSA-DT: A Malicious Traffic Detection Model Based on Graph Self-Attention Network and Decision Tree

Saihua Cai[✉], *Member, IEEE*, Han Tang, Jinfu Chen[✉], *Member, IEEE*, Tianxiang Lv, Wenjun Zhao, and Chunlei Huang

Abstract—Malicious attack has shown a rapid growth in recent years, it is very important to accurately detect malicious traffic to defend against malicious attacks. Compared with machine learning and deep learning technologies, graph convolutional neural network (GCN) achieves better detection results of malicious traffic due to additional consideration of the correlation between network traffic features. However, existing GCN-based detection models suffer from fixed weight assignment, only focusing on local features, lack the ability to model graph structure and relationships as well as having gradient disappearance. To solve these problems, this paper proposes the GSA-DT model based on graph self-attention network and decision tree. GSA-DT first preprocesses the original network traffic to obtain better traffic features and labels, and then uses GCN to extract the topological structure of network traffic as well as capture the correlation relationships among traffic features, where the ReLU activation function is replaced by LeakyReLU to overcome the problems of neuron “death” and gradient disappearance during the training process; It also introduces the self-attention mechanism into GCN to assign larger weights to the key features to reduce the interference of redundant features. Finally, GSA-DT uses decision tree to perform the detection of malicious traffic. Experimental results on four network traffic datasets show that GSA-DT model improves the detection accuracy over 1% on average than seven advanced malicious traffic detection models, and it also performs better in F1-measure, TPR, FPR as well as stability.

Index Terms—Malicious traffic detection, graph self-attention network, decision tree, LeakyReLU, deep learning.

I. INTRODUCTION

IN RECENT years, malicious attacks show a rapid growth trend, leading to severe service disruptions, thereby impacting the daily life of people. To mitigate the escalating security

risks, it is crucial to promptly detect the malicious attacks. Malicious attacks will generate malicious traffic, therefore, accurately detecting malicious traffic can find hidden attack behaviors, which can defend against network intrusion [1].

Previously, researchers mainly used statistics-based [2], port-based [3], signature-based [4] and other methods to detect malicious traffic. However, these methods show great limitations in face of unknown types of malicious traffic. Among them, statistics-based approaches have difficulty to identify malicious traffic that does not fit a known model or preset thresholds; Port-based approaches become ineffective due to the absence of fixed port numbers; Signature-based methods achieve low detection accuracy and robustness due to the use of techniques such as mutation, encryption and obfuscation in malicious attacks.

In recent years, machine learning (e.g., support vector machine [5], KNN [6], etc.) methods have been widely used in the field of malicious traffic detection, but the use of machine learning models has the following limitations: (1) It is very difficult in effectively identifying the complex patterns and nonlinear relationships in network traffic [7]; (2) It relies on additional preprocessing to deal with the problems included in complex network such as packet loss, retransmission and reordering [8]; (3) It has weak ability to recognize unknown attack types in complex tasks, has poor generalization ability and weak feature expression ability [9]. Compared with traditional machine learning methods, deep learning has stronger ability to handle the complex network traffic, stronger end-to-end learning ability, stronger ability to capture nonlinear relationships and can provide more robust and reliable detection results, therefore, deep learning techniques have gained widespread attention in recent years. However, traditional deep learning models (e.g., TCN, LSTM, RNN, etc.) do not consider the correlation between the features of network traffic in the model training process. In the network activities, the source IP and destination IP addresses as well as the source port and destination port have tight correlations, therefore, analyzing and modeling the correlation can improve the detection ability.

As a powerful deep representation learning method, GCN can learn node representation vectors that contain the information about nodes’ own features and associations with other nodes [10]. GCN shows remarkable performance in network analysis through constructing network traffic topology

Received 26 April 2024; revised 4 October 2024 and 16 December 2024; accepted 16 January 2025. Date of publication 12 February 2025; date of current version 22 April 2025. This work was partly supported by the National Natural Science Foundation of China (NSFC) (Grant nos. 62202206, U1836116, 62172194), the China Postdoctoral Science Foundation (Grant no. 2023T160275), the Natural Science Foundation of Jiangsu Province (Grant no. BK20220515), and the Qinglan Project of Jiangsu Province. The associate editor coordinating the review of this article and approving it for publication was M. Sayit. (Corresponding author: Jinfu Chen.)

The authors are with the School of Computer Science and Communication Engineering, and the Jiangsu Provincial Key Laboratory of Industrial Cyberspace Security Technology, Jiangsu University, Zhenjiang 212013, China (e-mail: caisaih@ujs.edu.cn; 222208016@stmail.ujs.edu.cn; jinfuchen@ujs.edu.cn; 222208039@stmail.ujs.edu.cn; 222208077@stmail.ujs.edu.cn; 2232208014@stmail.ujs.edu.cn).

Digital Object Identifier 10.1109/TNSM.2025.3531885

graphs as well as performing message passing and node updating on the graphs to capture the complex correlations among network traffic. For this reason, GCN models have been introduced in recent years for malicious traffic detection. However, existing GCN model [11] still has some shortcomings: (1) Assigning same weight to all features of network traffic, resulting in unable distinguish feature importance; (2) Only focusing on local features while ignoring global contextual information, resulting in cannot adequately capture the global dependencies of network traffic; (3) Lacking the ability to model graph structure and relationships, resulting in cannot well capture the dynamic nature in graph structure and complex relationships; (4) Having the problem of gradient disappearance due to using ReLU activation function, resulting in have limitation in nonlinear representation capability as well as capture the complex graph structure. These limitations affect the detection accuracy of GCN model.

In this paper, we propose a malicious traffic detection model called GSA-DT based on graph self-attention network and decision tree to solve these problems. The main contributions can be summarized as follows:

- We combine the GCN model with self-attention mechanism to achieve accurate detection of malicious traffic, this combination not only helps the detection model better learn the relationship between the features through GCN, but also helps extract the important features through self-attention mechanism and assign larger weights to them. In addition, we replace the ReLU activation function in the GCN model with nonlinear activation function LeakyReLU, which solves the gradient disappearance and neuron “death” problems of ReLU through its non-zero slope, thereby improving the detection performance.
- We leverage the interpretability of decision tree to improve the transparency of the model’s predictive reasoning; In addition, we utilize the strong generalization capability of decision tree to effectively handle the nonlinear relationships among network traffic features, thereby achieving the superior detection performance in complex network traffic environment.
- We introduce the Gini coefficient as a replacement for entropy in the decision tree and use it as the feature selection metric to measure the importance of features, thereby selecting the features much faster and further reducing the computation time; In addition, we also replace the Focal loss function in the decision tree with Cross-Entropy loss function to automatically pay more attention on a small number of hard-to-distinguish malicious traffic samples, thereby improving the detection performance.

The rest of this paper is organized as follows. Section II provides an overview of related works. Section III detailed describes the malicious traffic detection model called GSA-DT based on GCN and decision tree. Section IV gives and analyzes the experimental results. Section V discusses the potential threats that may affect the detection results. Finally, the whole paper is summarized and the future research directions are discussed in Section VI.

II. RELATED WORKS

Malicious traffic detection is a technique to effectively distinguish the malicious traffic from normal traffic, thereby maintaining the security of cyberspace. Initially, scholars used statistical-based [2], port-based [3], and signature-based [4] techniques to detect malicious traffic, but these traditional detection methods are difficult to effectively deal with increasingly sophisticated and stealthy network attacks, as well as difficult to adapt to new and unknown malicious attacks. With the rapid development of artificial intelligence, machine learning and deep learning techniques are widely used to effectively detect malicious attacks (including denial-of-service (DoS) attacks, malware infections, unauthorized access attempts, etc.) through learning their features.

Machine learning-based methods construct the machine learning models via learning the features of normal traffic and malicious traffic. Subsequently, the trained machine learning models are used to classify the collected network traffic, thereby effectively detecting malicious traffic and discovering malicious attacks. Wang et al. [5] proposed a cloud-based intrusion detection system called SCAEPSVM-IDS, but this algorithm lacks a detailed parameter optimization strategy. Machine learning-based methods usually require manually extracting the effective features from complex and variable network traffic; In addition, they cannot effectively extract the relevant features, resulting in a serious impact on the detection accuracy.

Unlike machine learning-based methods that require manual extracting the features, deep learning-based models have been widely studied in recent years due to their ability to automatically learn the features of network traffic and capture the complex patterns and rules associated with malicious behavior. For example, Haghnegahdar and Wang [12] proposed a model called WOA-ANN that uses artificial neural network to determine the optimal weights, it overcomes the drawbacks of tendency to fall into local minimal value, but this method suffers from high time complexity. Kan et al. [13] proposed an adaptive particle swarm optimized convolutional neural network (APSO-CNN) for IoT intrusion detection, it used particle swarm algorithm to optimize the structural parameters of a 1-D CNN by adaptive inertia weights, but its heuristic optimization algorithm had a long search time for offline models and redundant instances. Chiba et al. [14] introduced a backpropagation neural network (BPNN)-based optimization method and constructed an effective network intrusion detection system to effectively improve the detection accuracy, but these two methods can not effectively handle the temporal and sequential information. Chang and Branco [15] proposed two new graph neural network models called E-ResSAGE and E-ResGAT, which can significantly improve the detection accuracy on minority classes of data samples when dealing with class imbalance problem; However, these two models require higher computational complexity and training time. Huda et al. [16] introduced a fuzzy aggregation method using a modified density peak clustering algorithm (MDPCA) and deep belief network (DBN), it could adapt to dynamic changes of new malware, but it had a large time

complexity. Chen et al. [17] proposed a bi-directional temporal convolutional network-based malicious traffic detection model called BiTCN, it addressed the limitations of existing models and obtained high detection accuracy and better stability, but the bidirectional operation increases the training time of the model, especially on large-scale datasets. Venturi et al. [18] proposed a network intrusion detection system called ARGANIDS based on adversarial regularized graph self-encoder, it uses a GNN to capture the topology and traffic features, and adopts the adversarial training to make the model more robust to noisy and sparse data, thereby further improving the detection efficiency on complex network attacks; However, ARGANIDS relies on constructing detailed graph structures of network topologies, which may not be suitable for network environments with fast changing dynamics and large traffic sizes. Lin et al. [19] proposed a new multi-modal deep learning framework, it used raw byte and length sequences as the inputs and used a self-attention mechanism to learn the deep relationships between network packets in a stream. However, traditional deep learning models do not consider the correlations between the features of network traffic as well as ignore the topological information and contextual relationships of network traffic, as well as cannot fully consider the temporal and dynamically changing characteristics of data, resulting in a limited ability to process sequential data.

To solve the problems of traditional deep learning-based methods, scholars tried to introduce GCN models for malicious traffic detection. For example, Lan et al. [20] proposed a GraphSAGE-based model called E-minBatch GraphSAGE, it took the source ports of application layer and source IP addresses as source nodes, as well as the target ports of application layer and target IP addresses as target nodes, and the remaining traffic information as edge information to complete the construction of graph structure, but this model is limited in detecting industrial Internet traffic. Jiang et al. [21] designed a new trust assessment framework called GATrust based on the graph attention network, it assigned different attention coefficients to multifaceted attributes in the online social networks, thereby improving the prediction accuracy of social trust assessment, but its generality on graph structure data has not been tested. Zheng et al. [22] proposed an encrypted malicious traffic detection method called GCN-ETA based on graph convolutional neural network, it integrated a new traffic representation and an association construction scheme of network traffic; However, the flow direction of network traffic and the weights of different association types were not considered; In addition, this model also did not address the feature redundancy problem.

To solve these problems in existing GCN-based methods, this paper proposes the GSA-DT method based on graph self-attention network and decision tree to accurately detect malicious traffic. A comparison of the proposed GSA-DT and some reviewed malicious traffic detection methods is shown in Table I.

III. GSA-DT MODEL

In this section, we describe the malicious traffic detection model called GSA-DT based on graph self-attention network

TABLE I
COMPARISON OF MALICIOUS TRAFFIC DETECTION METHODS

Models	Activation function	Feature extraction	Support feature weighting
Wang <i>et al.</i> [5]	Sigmoid	Manual	×
Zhou <i>et al.</i> [11]	ReLU	Automatic	×
WOA-ANN [12]	Sigmoid	Manual	×
APSO-CNN [13]	Sigmoid	Automatic	×
Chiba <i>et al.</i> [14]	Sigmoid	Automatic	×
Huda <i>et al.</i> [16]	ReLU	Automatic	×
BiTCN [17]	ELU	Automatic	×
PEAN [19]	ReLU	Automatic	✓
E-minBatch GraphSAGE [20]	ReLU	Automatic	×
GATrust [21]	Softmax	Automatic	×
GCN-ETA [22]	ReLU	Automatic	×
GSA-DT (proposed)	LeakyReLU	Automatic	✓

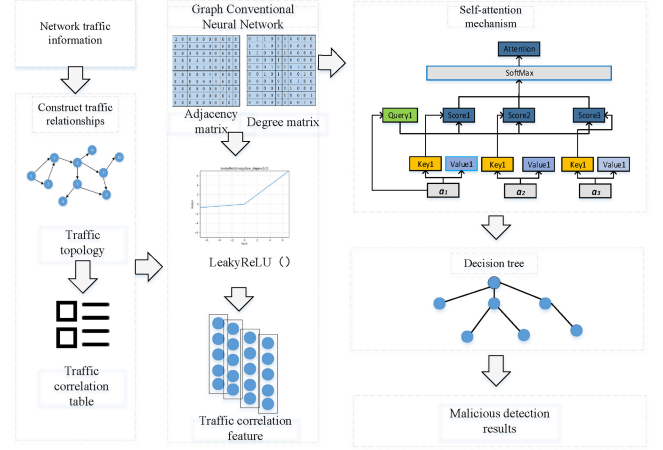


Fig. 1. The framework of GSA-DT model.

and decision tree from six aspects, including the general introduction of GSA-DT model, the traffic preprocessing, the construction of traffic topology relationships, the use of GCN model, the feature selection based on self-attention mechanism and the detection of malicious traffic based on decision tree.

A. General Introduction of GSA-DT Model

The GSA-DT first combines the GCN and self-attention mechanism to learn the complex relationship and feature representation of network traffic, and then combines the decision tree for malicious traffic detection. As shown in Fig. 1, the GSA-DT includes pre-processing phase, traffic topology relationship construction phase, GCN model construction phase, self-attention mechanism-based feature selection phase and decision tree-based detection phase.

(1) In the pre-processing phase, the process of traffic analysis, traffic cleaning, information extraction and traffic labeling provide accurate training samples and labels for malicious network traffic detection and analysis.

(2) In the traffic topology relationship construction phase, the address information is used to construct the relationship between traffic nodes and transform them into traffic adjacency matrix and traffic diagonal matrix.

(3) In the GCN construction phase, the topological structure information of network traffic graph is extracted to capture the feature relationships between network traffic; In addition, the LeakyReLU activation function is used to solve the gradient disappearance problem and neuron “death” problem of ReLU activation function.

(4) In the self-attention mechanism-based feature selection phase, we assign different weights to the features with correlations, this process ignores the features that contribute less to the correlation information of nodes and gives large weight to the features with stronger importance.

(5) In the decision tree-based detection phase, the filtered correlation features are classified to quickly and accurately detect malicious traffic.

B. Data Pre-Processing

The original network traffic is mostly in PCAP type, therefore, the original network traffic needs to be processed to make it suitable for GSA-DT model. In this paper, we process the original traffic through traffic analysis, traffic cleaning, information extraction and traffic labeling.

(1) The traffic analysis phase calls the Wireshark tool to analyze the original network traffic, thereby obtaining the statistical information of network traffic, such as the number of packets, source IP and destination IP statistics, protocol distribution, etc.

(2) The traffic cleaning phase calls the statistical information as well as removes the empty and duplicate information, thus improving the generalization ability of the model.

(3) The information extraction phase mainly extracts the feature information in network traffic, including: source IP address, destination IP address, incoming and outgoing traffic packet size, incoming and outgoing traffic packet number, traffic packet number, port number and other information. We select these features because empirical analysis shows that in malicious behavior detection, the abnormal IP addresses, unconventional port usage, and a large number of small traffic packets may indicate the malicious activities. And then, the string information is segmented using Pandas library and written the traffic information in the form of Dataframe.

(4) The traffic labeling phase aims to label the traffic detection information (including: normal traffic, malicious traffic, and malicious traffic categories) according to the traffic type, thereby providing accurate training samples and labels for malicious traffic detection and analysis. In addition, the obtained traffic information is stored in the form of tables to obtain the suitable form for detection model.

C. Constructing Traffic Topology Relationships

In the phase of constructing traffic topology relations, it is necessary to construct traffic tables that have undergone preprocessing operations to obtain traffic topology maps, thereby facilitating the detection of malicious traffic. After obtaining the traffic information, different nodes are connected with the source IP address as starting point and the destination IP address as ending point. At the same time, each different IP node is numbered to form the network topology. The information carried by connected edges on the graph is the characteristic information, including: source IP address, destination IP address, traffic packet size, traffic packet number, port number, etc.

The traditional methods mainly construct graph structures based on the relationship between nodes and edges in the

TABLE II
AN EXAMPLE OF NETWORK TRAFFIC

ID	ip.src	ip.dst	in_packets	in_bytes	out_packets	out_bytes	duration
1	10.0.2.15(1)	173.194.10.145(2)	8615	9386538	1605	118806	117.7438
2	10.0.2.15(1)	216.58.211.3(3)	6899	675475	2558	258818	746.0223
3	216.58.211.3(3)	173.194.10.174(6)	6998	9283490	1179	81772	941.4655
4	173.194.10.174(6)	173.194.10.215(9)	488	843666	1112	88014	204.1952
5	173.194.10.174(6)	151.101.13.164(5)	3040	5238664	884	77714	97.5009
6	173.194.10.174(6)	216.58.211.4(7)	4115	4456484	1195	121187	910.2499
7	151.101.13.164(5)	216.58.207.6(4)	4123	4490208	1054	78861	642.7948
8	216.58.211.4(7)	93.184.220.70(8)	4276	4578176	786	70310	48.9309
9	151.101.13.164(5)	93.184.220.70(8)	3386	3562875	881	81427	193.724
10	216.58.211.4(7)	216.58.205.28(10)	3304	3479344	881	71956	449.8705
11	173.194.10.145(2)	216.58.211.3(3)	1838	2106978	591	70916	256.9416

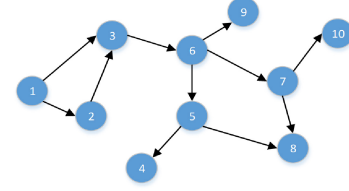


Fig. 2. The traffic topology relationships diagram.

network and extract relevant features to describe the properties and behaviors of the nodes and edges, but these methods lack classification by nodes, and thus cannot capture the differences and correlations between the network traffic generated by different nodes. For this reason, we improve it through classifying network traffic in the graph by node and treating the network traffic generated by same node as the same traffic type, thus transforming the problem of malicious traffic detection into malicious node detection. Specifically, when constructing a network traffic graph, each node represents a network entity, such as an IP address or a device. In order to better detect malicious traffic, we add labels representing the traffic type of each node. By adding labels to the nodes, the model can easily distinguish traffic and identify different types of attacks. Among them, the traffic label of each node can be set based on its communication behavior and specific features, and the label can cover different types of malicious traffic, thus, the traffic type is identified by adding an attribute to each node.

Taking the network traffic shown in Table II as an example. Assuming that source IP 10.0.2.15 is node 1 (as marked in the parentheses), and its destination IPs are 173.194.10.145 (noted as node 2) and 216.58.211.3 (noted as node 3), there is an association between node 1 and nodes 2 and 3 in the topology, as illustrated by the traffic topology in Fig. 2. In this network traffic graph, each node (such as nodes 1, 2 and 3) represents a network entity.

To improve the detection accuracy of malicious traffic, we assign specific traffic type labels rather than general “malicious” or “benign” label to each node to represent different types of traffic. For example, if node 1’s behavior resembles normal communication patterns, it can be labeled as “benign traffic”; However, if node 2 shows the characteristics of a DDoS attack, it would be labeled as “DDoS”, and if node 3’s traffic matches the patterns of malware distribution, it would be labeled as “malware”. These specific labels not only help the model distinguish between benign traffic and malicious traffic, but also identify the exact type of malicious behavior, such as DDoS, phishing, malware, etc.

After constructing the traffic topology, the adjacency matrix B and diagonal matrix D are obtained according to the

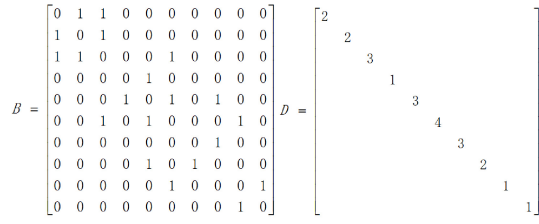


Fig. 3. The adjacency matrix and diagonal matrix of network traffic.

constructed diagram of traffic topology relationships. The value of $B_{i,j}$ in B is 0 or 1, where 0 means there is no connection between two nodes and 1 means two nodes have network communication. Take node 1 as an example, it has network communication with node 2 and node 3, then $B_{1,2}$, $B_{2,1}$, $B_{1,3}$ and $B_{3,1}$ are recorded as 1. Because there is no connection between node 1 and node 4, $B_{1,4}$ and $B_{4,1}$ are recorded as 0. Matrix D is used to represent the number of neighboring nodes connected with each node, and the number of neighboring nodes is added 1 when there is a network communication between node, where $D_{i,i}$ denotes the number of neighboring nodes connected to node i . Since node 1 is adjacent to node 2 and node 3, $D_{1,1}$ is recorded as 2. The final adjacency matrix B and diagonal matrix D obtained from the network traffic listed in Table II are shown in Fig. 3.

D. Graph Convolutional Neural Network

After obtaining the adjacency matrix and diagonal matrix, it is necessary to construct a structure to process them. Currently, the GIN [23], GraphSAGE [20] and GCN [24] are often used as the backbone of the model. Among them, a key property of GIN model is that the aggregation of node features is invariant, but network traffic is the packets transmitted from one place to another over a period of time, this process is dynamic, thus, GIN model is not applicable to our scenario; GraphSAGE only considers the aggregation between nodes and neighboring nodes but ignores the internal correlation between the features, resulting in not being able to capture the global topology information well. Compared with these two models, GCN can better learn the information of neighboring nodes around each node, can learn the parameters for different node locations in a single training through parameter sharing, as well as can better balance the computational complexity and expressive power by means of feature propagation and convolutional operations. Therefore, we choose GCN as the backbone of the model.

GCN is a multi-layer convolutional neural network, it learns from the attributes between the nodes in the established graph and the neighboring nodes to obtain a new feature representation for each node. To improve the computational efficiency, matrixs B and D need to be normalized using Equation (1) to obtain the normalization matrix S .

$$S = D^{-1/2} B D^{-1/2}. \quad (1)$$

After sending S as input to 1-D convolutional network, the first-order neighbor association feature H_1 is learned by

Equation (2), where σ denotes the activation function and θ_1 is the initial weight matrix.

$$H_1 = \sigma(S\theta_1). \quad (2)$$

The features of network traffic often show the complex nonlinear patterns, retaining the nonlinear mapping capabilities can help to better capture these complex nonlinear patterns, thereby improving the detection accuracy. Therefore, it is required to use activation function with stronger nonlinear expressivity to better fit such nonlinear features. GCN usually uses ReLU activation function, but use of ReLU has the following problems: (1) The output is zero when the input is negative (that is, appearing neuron “death” problem), resulting in some neurons being unable to update the corresponding weights during the training process due to the cessation of learning; (2) Leads to the problem of gradient disappearance, especially the gradient may become very small in the backpropagation process. To solve these problems, scholars replaced it with Sigmoid [25], ELU [17] and Tanh [26] in recent years. In contrast, the LeakyReLU activation function retains the linear properties of ReLU at positive inputs, which effectively avoids the problem of gradient vanishing; It is also able to maintain a more stable gradient update rate, avoiding almost no effective gradient update in the first few layers like Tanh. These advantages of LeakyReLU motivate us to use it instead of ReLU to achieve better detection result.

After acquiring the features, the deep features are obtained by stacking multiple convolutional networks to get a larger perceptual field. For the k^{th} convolutional layer, its features H_k are nonlinearized from the features obtained from previous $(k - 1)$ layer using LeakyReLU. The details are shown in Equation (3).

$$H_k = \sigma(SH_{k-1}\theta_k). \quad (3)$$

During the training process of GCN model, the graph association relationship representation is achieved by learning its association relationship to obtain the feature information of different nodes with their neighbor nodes and edge information. At the same time, with the help of sharing same network node structure information, the features of each node for each global inter-neighbor node are obtained under the action of multi-layer convolution, thereby obtaining the spatial structure characteristics of network traffic.

E. Self-Attention Mechanism

Although GCN has strong ability in feature learning of network traffic, but it does not give more attention to the features that reflects malicious traffic, which leads to unstable detection result, it is necessary to assign different weights to the features according to their importance in the feature screening process. The self-attention mechanism reduces the impact of unimportant and redundant features [27], [28], [29] on malicious traffic detection by assigning larger weights to the features that have a greater impact on malicious traffic detection and smaller weights to these unimportant features. Based on the advantage of self-attention mechanism, this paper improves the GCN model via adding the self-attention

mechanism layer, thereby further improving the detection ability of GCN model.

The self-attention mechanism obtains the relevance of different traffic inputs through computing the attention coefficients on different relevant inputs, and then assigns the low weights to the irrelevant features to retain important vector features. The overall structure of self-attention mechanism is shown in the upper right corner of Fig. 1, it is mainly composed of Query, Key and Value. Firstly, it inputs the n feature vectors h obtained by GCN into self-attention model, and then obtains the traffic feature matrix L through stitching the features via the input layer. The specific process is shown in Equation (4).

$$L = [h_1, h_2, \dots, h_n]. \quad (4)$$

Meanwhile, L is input into the embedding layer to map it to three different spaces to obtain Q , K and V matrices. This process is shown in Equation (5), where W_Q , W_K and W_V represent the randomly obtained space matrices, respectively.

$$Q = W_Q * L, K = W_K * L, V = W_V * L. \quad (5)$$

After obtaining three different spatial matrices, the similarity between each query vector q_i and each different traffic vector is calculated to obtain the linkage score (recorded as *Score*). The used similarity measure is shown in Equation (6), where the *Score* is obtained by calculating the product of each vector's own q and k and then summing them. Finally, the weight relationship matrix A of L is obtained by stitching the *Score*, which is shown in Equation (7).

$$\text{Score}_i = \sum_{i=1}^n q_i * k_i. \quad (6)$$

$$A = [\text{Score}_1, \text{Score}_2, \dots, \text{Score}_n]. \quad (7)$$

And then, the Softmax function is used to normalize A to obtain similarity matrix A' , thus reducing the computational difficulty, which is shown in Equation (8).

$$A' = \text{Softmax}(A) = \frac{e^{s_i}}{\sum_{j=1}^n e^{s_j}}. \quad (8)$$

Finally, the attention output matrix O is calculated by weighting and summing the key value matrix V based on A' , which is shown in Equation (9).

$$O = V * A'. \quad (9)$$

F. Decision Tree

After getting the feature vector O , it is required to use feature vector to determine whether the network traffic is malicious traffic. For the detection of malicious traffic, the interpretability is very important, but it is a common problem encountered by deep learning models. The introduction of GCN allows the model to learn more informative and discriminative features that can be used to improve the detection accuracy. To enhance the interpretability of detection models, it is better to integrate a machine learning model to provide a transparent and interpretable framework for distinguishing malicious traffic from normal traffic, thereby facilitating the

interpretability of the predictive reasoning process. Compared with other machine learning models, decision tree has the advantages of small computational effort, easy transformation of classification rules, high accuracy and strong generalization [30], where the internal nodes represent the characteristic features of network traffic and the leaf nodes represent its traffic classification labels.

In the detection phase, the obtained associated traffic features are input and the loss value is calculated via loss function, thereby obtaining the parameters according to this value. Currently, the commonly used loss functions include: mean square error (MSE), mean absolute error (MAE), focal loss and cross-entropy loss. The malicious traffic detection belongs to multi-classification research, it requires GSA-DT model to be able to automatically detect malicious traffic by learning the features of network traffic, therefore, focal loss is not suitable due to the need of manually adjusting the weight parameters and the lack of automation capability. In addition, the scale of malicious traffic generated by malicious attacks is much smaller compared to that of normal traffic generated by normal accesses, i.e., there is a data imbalance in the network traffic, which leads to the fact that both MSE and MAE are not suitable for our research scenario due to their excessive focus on the main category (normal traffic). Compared with these commonly used loss functions, the cross-entropy loss can automatically focus the attention on a small number of hard-to-distinguish malicious traffic samples, which improves the detection ability of the model. For this reason, it is chosen as the loss function in the decision tree model. The loss function of cross-entropy is shown in Equation (10), where p_i represents the probability vector of predicted classification results, y_i represents the label category of actual samples. Finally, the corresponding parameters are updated through backward derivation.

$$\text{Loss} = - \sum_{i=1}^K y_i \log(p_i). \quad (10)$$

Both Gini coefficient and entropy are commonly used feature selection metrics to measure the importance of features and they have different properties in decision trees. The Gini coefficient is a measure of impurity that measures the probability that two samples are randomly selected from a dataset and their category labels do not match. The smaller Gini coefficient indicates the lower impurity of the dataset and the higher importance of the feature. The process of Gini coefficient calculation does not involve logarithmic operations, thus, the computation is more efficient. Entropy is another commonly used feature selection metric, which measures the degree of information clutter in a dataset. The calculation of entropy requires logarithmic operations, which requires more computational time compared to the Gini coefficient. In the case of large-scale datasets and complex feature spaces, the Gini coefficient can perform feature selection faster and reduce the computation time. Therefore, we use the Gini coefficient (shown in Equation (11)) as a feature selection function, where k represents the number of traffic categories and p_i represents the probability that the traffic belongs to i^{th}

category. The classification result is measured by the value of Gini coefficient, and the classification of network traffic is performed based on the rank of calculated Gini coefficients.

$$\text{Gini}(D) = \sum_{i=1}^k p_i(1 - p_i) = 1 - \sum_{i=1}^k p_i^2. \quad (11)$$

In general, the proposed GSA-DT model uses GCN network as the main body to fully consider the correlation features of network traffic as well as adopts the LeakyReLU activation function to solve the neuron “death” problem of ReLU and thus improving the expressiveness of GCN model. And then, it uses a self-attention mechanism to provide larger weight for important features as well as reduce the influence of redundant features; In addition, it uses decision tree to classify the network traffic, thus improving the detection accuracy of malicious traffic while providing the model a stronger generalization capability. With these improvements, the proposed GSA-DT model can effectively capture the features of global network traffic by learning the differences and correlations between different types of network traffic, thereby accurately detecting malicious traffic.

In summary, our model utilizes the GCN and self-attention mechanism to effectively enhance the feature representation, which in turn achieves interpretability through decision tree. Such integration aims to ensure effective detection of malicious traffic while balancing performance and interpretability.

G. The Workflow of GSA-DT Model

The flowchart of GSA-DT is shown in Fig. 4, it is composed feature pre-processing phase, feature extraction phase and malicious traffic detection phase.

(1) In the feature pre-processing phase, network traffic will undergo a series of operations including traffic analysis, cleaning and labeling to ensure that it can be used in the correct format and structure for subsequent analysis; One of the key tasks in this phase is to construct the topology of network traffic using the source and destination IP addresses, and then generate the adjacency and diagonal matrices to better represent the relationships between traffic nodes.

(2) In the feature extraction phase, GCN is used to extract the topological structure of network traffic, allowing the model to capture the intrinsic correlations between the features; And then, the LeakyReLU is adopted to replace the traditional ReLU to solve the gradient vanishing and “neuron death” problems. In addition, the model also introduces a self-attention mechanism to focus on the key features and assign large weights to important features, thereby reducing the impact of redundant features. These improvements improve the model’s ability to distinguish between normal and malicious traffic.

(3) In the malicious traffic detection phase, the decision tree with Gini coefficient as feature evaluation index is used to classify the extracted features, thereby efficiently and accurately detecting malicious traffic while reducing false positive and false negative rates, and providing good interpretability.

The detailed process of malicious traffic detection using GSA-DT model is shown in Algorithm 1. Firstly, the network

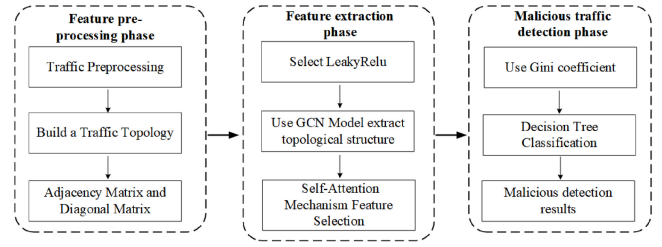


Fig. 4. The flowchart of GSA-DT model.

Algorithm 1 GSA-DT

Input:

Network traffic NT , learning rate, hidden, epoch, dropout

Output:

Detection result R

```

1: preprocess  $NT$  to obtain features  $X$ 
2: Initialize parameters (learning rate, hidden, epoch, dropout)
3: construct traffic topology relationships  $G$ 
4:  $G \rightarrow$  GCN model
5: for each epoch in  $R$  do
6:   for each hidden in  $R$  dataset do
7:     extract traffic features according to learning rate and dropout
8:     classification result  $\leftarrow$  label
9:     calculate loss function
10:    renew weight  $W$ 
11:    obtain traffic correlation feature  $H$ 
12:   end for
13:   assign features' weight  $W'$  using self-attention mechanism
14: end for
15:  $R = DT(H, W')$ 
16: return  $R$ 
  
```

traffic is preprocessed and the relationship between network traffic node is constructed using the address information of traffic to get the key information and feature annotations of network traffic. Based on the obtained network traffic information, the contained address information is extracted and the traffic topology relationship graph is constructed based on the source IP address and destination IP address and other information, thereby obtaining the adjacency matrix and diagonal matrix of network traffic. And then, the features are extracted using GCN for the constructed network traffic graph structure, and the features with stronger correlation are obtained by giving large weight to the important traffic features and ignoring the features that contribute less to the node correlation information through self-attention mechanism. In addition, the ReLU activation function in the GCN model is replaced by the LeakyReLU, thus improving the expressiveness of the model as well as making the model to have better properties when handling negative anomalies and preserving negative gradients. Next, the filtered correlated features are classified in combination with the decision tree, thus detecting malicious traffic more accurately. Finally, the test set is input to the GSA-DT model to verify its detection effect.

IV. EXPERIMENT RESULTS AND ANALYSIS

We compare GSA-DT model with seven malicious traffic detection models on four network traffic datasets to test its efficiency. Experiments are conducted on a Linux 64-bit OS

with Intel Xeon Gold 5218R CPU and NVIDIA GeForce RTX 3090 GPU, using Python (Python version 3.8) as the AI library.

A. Description of Network Traffic Datasets

To verify the detection efficiency of GSA-DT model, we use four network traffic datasets (USTC-TC2016, Stratosphere, CTU and UJS-IDS2024) in the experiments, the distribution of these network traffic datasets is shown in Table III, where all normal network traffic are classified into “Normal”.

(1) USTC-TFC2016 [31] contains ten types of malicious traffic and one normal traffic (composed of ten categories of normal traffic) collected by the simulated devices from 2011 to 2015, the format is pcap file and the labels of traffic types are provided.

(2) Stratosphere [32] is a network traffic dataset with a mixture of encrypted and non-encrypted traffic, it includes a series of network IPS/IDS traffic, such as Zeus and botnets. Because the scale of malicious traffic is large than normal traffic in this dataset, we integrated some normal traffic in CTU-13 dataset into this dataset to balance the scale of network traffic.

(3) CTU [33] is a network traffic dataset collected by the devices from 2016 to 2019, it contains ten types of malicious network traffic and one type of normal network traffic.

(4) UJS-IDS2024 is a network traffic dataset collected by multiple normal software and malware in the real environment by the Key Laboratory of Industrial Cyberspace Security Technology of Jiangsu University from March 5, 2024 to April 18, 2024.

In the experiments, we follow the 90%/10% ratio to divide each network traffic dataset into a training set and a testing set, where 90% of the data is used to train the model and the remaining 10% is used to evaluate the performance of the model. This segmentation method ensures that the model can be trained on sufficient data, while also evaluates its generalization ability through unseen data.

B. Evaluation Metrics

In order to evaluate the efficiency of GSA-DT model, we use four widely used evaluation metrics including TPR, FPR, F1_measure and Accuracy in the experiments [17], [19], [34], [35], they are calculated as shown in Equations 12 to 15. These four metrics can well evaluate GSA-DT model from different perspectives. Among these equations, TP (True Positive) is the number of malicious traffic correctly identified by GSA-DT, FP (False Positive) is the number of normal traffic incorrectly identified as malicious by GSA-DT, FN (False Negative) is the number of malicious traffic not correctly identified by GSA-DT, TN (True Negative) is the number of normal traffic correctly identified by GSA-DT.

(1) TPR (True Positive Rate): It indicates the number of correctly-detected malicious network traffic to the total number of malicious network traffic samples, which is calculated by Equation (12).

$$TPR = \frac{TP}{TP + FN} \quad (12)$$

(2) FPR (False Positive Rate): It indicates the number of mis-detected malicious network traffic to the total number of normal network traffic samples, which is calculated by Equation (13).

$$FPR = \frac{FP}{(TN + FP)} \quad (13)$$

(3) F1_measure: It is the harmonic mean of precision and recall with a maximum of 1 and a minimum of 0, which is calculated by Equation (14).

$$F1_measure = \frac{2 \times \frac{TP}{TP+FP} \times \frac{TP}{(TP+TN)}}{\frac{TP}{TP+FP} + \frac{TP}{(TP+TN)}} \quad (14)$$

(4) Accuracy: It indicates the ratio of the number of accurately detected network traffic samples to the total number of network traffic samples, which is calculated by Equation (15).

$$Accuracy = \frac{TP + TN}{TP + FP + TN + FN} \quad (15)$$

C. Baselines

To evaluate the efficiency of GSA-DT model, seven baselines are used in the experiments.

(1) E-minBatch GraphSAG [20]: It is a graph neural network for malicious traffic detection, it represents the association relationships between the features of network traffic by constructing a graph structure and performs malicious traffic detection by aggregating the information of neighboring nodes.

(2) GATrust [21]: It is a graph neural network model based on graph attention mechanism, it integrates multiple attributes (including user context-specific information, network topology information and locally generated social trust relationships) of users, and assigns different attention coefficients to the attributes of users in online social networks, thereby improving the prediction accuracy.

(3) GCN-ETA [22]: It is an encrypted malicious traffic detection model based on GCN, it improves the traditional GCN by considering the statistical features of network flows (internal information) and the structural information between network flows (external connections), thereby effectively improving the detection efficiency.

(4) TCN [36]: It is a malicious traffic detection model based on temporal convolutional network, it extracts the key features using the features of byte sequences in network traffic packets as well as avoids neurons “death” by changing the ReLU to ELU, thereby obtaining high detection performance.

(5) XGBoost [37]: It is an ensemble malicious traffic detection model based on gradient boosting decision tree, it also uses WOA (Whale Optimization Algorithm) to seek for the best parameters.

(6) BiTCN [17]: It is a malicious traffic detection model based on bidirectional temporal convolution network (BiTCN), it uses TCN model to better grasp the sequence characteristics of network traffic as well as improves the original one-way model to a two-way model to capture more characteristics of network traffic.

(7) PENA [19]: It is a packet-level end-to-end attentive network, it treats original byte and length sequences as inputs

TABLE III
THE INTRODUCTION OF ORIGINAL NETWORK TRAFFIC

USTC-TFC2016			Stratosphere			CTU			UJS-IDS2024		
Categories	Training Set	Test Set	Categories	Training Set	Test Set	Categories	Training Set	Test Set	Categories	Training Set(KB)	Test Set(KB)
BitTorrent	6765	752	Zeus	5320	1330	Ursnif	23180	2576	ArkeStealer	143529	15947
Zeus	9873	1097	Miuref	10750	2677	Hibot	41292	4588	AsyncRAT	24742	2749
Virut	29793	3995	Dreambot	10670	2650	Torgan	6425	714	AveMariaRAT	267781	29753
Tinba	7654	850	Gookit	6270	1770	Tinba	46206	5134	Formbook	341539	37948
Miuref	12133	1348	Dridex	8869	2218	Zeus	16640	1849	Hive	504310	56034
Shifub	8667	963	Trickbot	10379	2599	Artemis	22824	2536	Kutaki	210786	23420
SMB	35046	3894	Ursnif	5850	1463	Coimminer	4979	553	Loki	15534	1726
Nsis	5463	607	Hancitor	9110	2274	Dridex	8894	988	MassLogger	154181	17131
Neris	30411	3379	Normal	106111	26220	Miuref	24543	2727	OskiStealer	69631	7736
Cridex	14747	1639				Trickbot	8858	984	RedLineStealer	144054	16006
Geodo	36853	4095				Normal	34246	3805	Normal	1057049	117449
Hibot	5733	637									
Normal	75238	8359									

and uses the self-attention mechanism to learn the deep relationships between network packets.

D. The Selection of Parameters

For the proposed GSA-DT model, the parameters including the number of hidden units, dropout and epoch number will affect its detection efficiency. To obtain better detection accuracy, we test the efficiency of GSA-DT under these parameters on three widely used public network traffic datasets, where the used loss function is LeakyReLU. The experimental results are shown in Fig. 5-Fig. 7.

As can be seen from Fig. 5 that GSA-DT has best performance and smoother fluctuations when the number is selected as 16 on three datasets. When the number is select less than 16, the performance is worse; With the increase number of hidden units, the performance of GSA-DT shows a decrease trend, which is caused by the overfitting. After a comprehensive comparison, the number of hidden units is chosen to be 16. As is shown in Fig. 6 that on USTC-TC2016 and Stratosphere datasets, the performance of GSA-DT is low when the dropout is chosen as less than 0.05, it is owing to that the lower dropout causes the model to be overly dependent on specific neurons, making the network less expressive and prone to overfitting. In contrast, when the dropout is chosen as 0.06 and 0.07, the performance is lower than that when it is chosen as 0.05, the reason is that large dropout causes many neurons to be randomly discarded, thus losing a large amount of information. On the CTU dataset, the performance of GSA-DT model is higher when the dropout is chosen as 0.05. After a comprehensive comparison, the dropout is chosen as 0.05. As is shown in Fig. 7 that compared with seven detection models, GSA-DT has the best detection accuracy at different epochs, this is because it considers the correlation between traffic features and combines self-attention mechanism to better capture the features. It can be seen from Fig. 7(a) and Fig. 7(b) that when the epoch number is less than 5, GSA-DT shows a growth trend as epoch number increases, but there is a significant decline with the further increase of epoch number. When the epoch number is larger than 8, GSA-DT has stable trend, and it has better performance when the epoch number is set to 10. Therefore, the epoch number is chosen as 10.

E. Detection Results of GSA-DT Model

To validate the performance of GSA-DT model, we conducted a large number of experiments on four network traffic

datasets and compared the results with seven state-of-the-art models. The average experimental result is shown in Table IV.

The experimental results in Table IV show that on the USTC-TFC2016 dataset, GSA-DT has better performance than seven compared models, and its average detection accuracy can reach to 99.96%, while the GCN-ETA model has the worst performance of 97.50%. In contrast, the TCN and XGBoost only consider the attribute statistical features of network traffic without considering the relationship between network traffic, thus, their detection accuracy is slightly lower than that of GSA-DT model. Compared with GATrust, a graph-based malicious traffic detection model with better detection results, the GSA-DT model has improved the detection accuracy by about 1.54%, TPR by about 1.89%, and F1-measure by about 1.92%, and decreased the FPR by about 0.36%. In addition, the GSA-DT model has the least fluctuation in detection efficiency. For the Stratosphere dataset that contains encrypted network traffic, the detection effectiveness of all eight models shows a decrease trend, but GSA-DT still performs best among all models, which can reach to about 98.49%, while the accuracy of other detection models is 96.5% on average. Because the encrypted traffic hides the attack information through encryption protocols, it is difficult to extract the statistical features of network traffic, which leads to the poor detection performance of TCN and XGBoost models because they do not use relevant features. Compared with them, three graph-based detection models (including GCN-ETA, E-minBatch GraphSAGE and GATrust) improve the detection efficiency in some metrics, but they have lower TPR due to the fact that not use a self-attention mechanism to assign larger weight to important features. Compared with GCN-ETA, E-minBatch GraphSAGE and GATrust, GSA-DT has improved accuracy by about 0.98%, 2.18% and 2.15%, TPR by about 2.67%, 1.41% and 0.96%, F1-measure by about 1.35%, 1.58% and 1.38%. The experimental results show that GSA-DT performs well on the encrypted network traffic via learning the correlation relationships and assigning differential weights to features. The experimental results on CTU dataset show that GSA-DT has better performance than the compared models. In the metrics of accuracy, TPR and F1-measure, GSA-DT improves about 3.03%, 3.92% and 3.4% over E-minBatch GraphSAGE model, about 1.69%, 2% and 2.02% over GATrust model, about 3.84%, 2.92% and 2.46% over GCN-ETA model, about 0.92%, 1.29% and 1.09% over TCN model, about 1.5%, 2.83% and 2.41% over XGBoost model. Compared with BiTCN and PENA, GSA-DT has improved accuracy by about 0.76% and 0.35%, TPR by about 0.73% and 0.51%, F1-measure by about

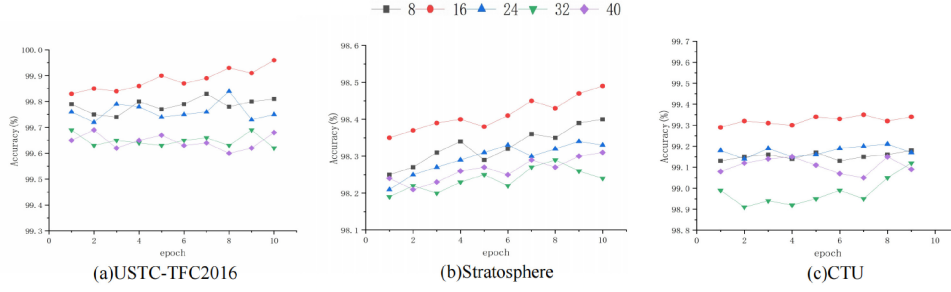


Fig. 5. The detection accuracy of GSA-DT under different number of hidden units.

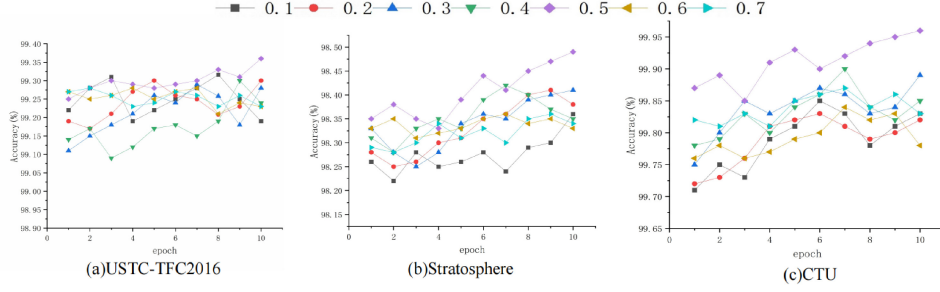


Fig. 6. The detection accuracy of GSA-DT under different dropout.

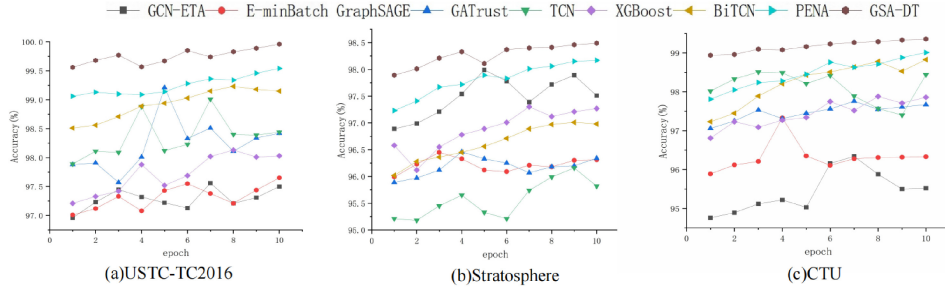


Fig. 7. The detection accuracy of GSA-DT under different epoch number.

TABLE IV
AVERAGE DETECTION EFFICIENCY OF EIGHT MALICIOUS TRAFFIC DETECTION MODELS (%)

Datasets	Models	E-minBatch	GraphSAG	GATrust	GCN-ETA	TCN	XGBoost	BiTCN	PENA	GSA-DT
USTC-TFC2016	Accuracy	97.65±0.76		98.42±0.54	97.50±0.51	98.44±0.42	98.03±0.21	99.23±0.42	99.54±0.22	99.96±0.13
	TPR	97.87±0.82		98.06±0.41	99.28±0.71	98.71±0.35	97.68±0.65	99.22±0.13	99.37±0.14	99.98±0.08
	FPR	0.57±0.10		0.40±0.08	0.25±0.05	0.31±0.05	0.61±0.07	0.23±0.04	0.14±0.03	0.04±0.02
	F1-measure	97.53±0.77		98.34±0.33	98.38±0.34	98.58±0.27	97.59±0.67	99.18±0.13	99.30±0.11	99.96±0.09
Stratosphere	Accuracy	96.31±0.62		96.34±1.02	97.51±0.47	95.82±0.67	97.27±0.25	97.01±0.24	98.17±0.15	98.49±0.16
	TPR	96.41±0.55		96.86±0.93	95.15±0.86	96.34±0.83	96.87±0.63	97.06±0.21	97.74±0.20	97.82±0.18
	FPR	2.72±0.41		2.03±0.30	2.91±0.37	2.90±0.42	2.60±0.35	2.30±0.32	1.88±0.30	1.64±0.30
	F1-measure	96.37±0.61		96.57±0.88	96.60±0.66	95.23±1.01	96.73±0.77	97.10±0.24	97.65±0.26	97.95±0.23
CTU	Accuracy	96.33±0.43		97.67±0.51	95.52±0.35	98.44±0.52	97.86±0.36	98.60±0.40	99.01±0.10	99.36±0.15
	TPR	95.78±1.51		97.70±0.89	96.78±0.44	98.41±0.39	96.87±0.43	98.97±0.20	99.19±0.10	99.70±0.14
	FPR	2.70±0.36		1.67±0.20	2.11±0.24	1.14±0.13	2.31±0.21	1.06±0.10	0.52±0.08	0.32±0.03
	F1-measure	96.27±0.52		97.65±0.67	97.21±0.31	98.58±0.63	97.26±0.34	98.83±0.10	99.15±0.10	99.67±0.22
UJS-IDS2024	Accuracy	91.46±0.91		96.10±0.87	90.56±0.77	92.10±0.91	95.85±1.24	93.31±0.83	96.28±0.35	96.50±0.20
	TPR	92.83±1.01		96.58±0.47	91.53±0.96	92.39±0.36	95.91±0.63	93.55±0.29	96.87±0.43	96.90±0.40
	FPR	6.99±0.41		4.17±0.28	7.98±0.24	6.23±0.40	4.73±0.27	5.16±0.38	3.66±0.26	3.41±0.21
	F1-measure	92.06±1.12		96.5±0.68	91.41±1.07	92.11±0.82	95.87±0.66	93.24±0.58	96.65±0.25	96.72±0.20

0.84% and 0.52% respectively, and decreased the FPR by about 0.74% and 0.2%. On the UJS-IDS2024 dataset, the GSA-DT model improved about 5.04%, 4.07% and 4.66% over EminBatch GraphSAGE, about 0.4%, 0.32% and 0.22% over GATrust in terms of accuracy, TPR and F1-measure. In summary, the traditional deep learning-based detection models have relative low performance for not considering

the correlation between features, and the traditional graph-based detection models improve the detection accuracy due to they learn more features through the additional consideration of correlation between traffic features. However, graph-based models do not perform well in detecting small classes of malicious traffic. Compared with seven baselines, the proposed GSA-DT model exhibits excellent performance and achieves

TABLE V
TIME COST OF GSA-DT MODEL UNDER DIFFERENT SIZES OF
UJS-IDS2024 DATASET (SEC.)

Data Size	2000	6000	10000	20000	50000	100000	200000
E-minBatchGraphSAGE	1.44	6.56	15.36	35.65	106.51	232.44	492.11
GATrust	3.31	15.83	22.48	55.25	136.97	301.61	533.54
GCN-ETA	1.27	5.81	14.89	34.22	98.51	219.36	451.31
TCN	0.96	4.68	7.99	12.21	36.53	79.68	168.56
XGBoost	1.05	5.84	8.42	20.68	74.75	183.62	244.31
BiTCN	1.08	5.13	8.54	14.37	41.68	87.23	187.62
PEAN	1.43	6.35	14.43	33.29	102.15	215.27	446.83
GSA-DT	1.21	6.01	14.21	32.55	95.72	211.42	435.77

significant improvements in all evaluation metrics, which can be attributed to the following factors: (1) The introduction of self-attention mechanism makes it assigning large weights to the important features, which leads it to reduce the impact of redundant features on malicious traffic detection; (2) The use of decision tree makes GSA-DT having the advantages of good multi-classification capability, robustness to missing data and outliers, and efficient feature selection and training; (3) The use of LeakyReLU solves the neuron “death” problem and gradient disappearance problem of ReLU.

F. Stability Analysis of GSA-DT Model

In addition to the detection efficiency, the stability of detection model is another important index. In this subsection, we compare the stability of GSA-DT with seven detection models on four network traffic datasets, and the experimental result is shown in Fig. 8.

As is shown in Fig. 8 that on these datasets, GSA-DT model has highest detection accuracy, TPR and F1-measure as well as lowest FPR compared with seven detection models, and it also has no outlier in detection results as well as has minimal gap between the upper and lower quartiles. In contrast, the machine learning-based XGBoost model has many outliers due to it tends to overfit the real data as well as ignores the correlation between the contexts of network traffic. Because TCN model considers the temporal characteristics of network traffic during the training process, it shows higher performance in terms of accuracy, but it ignores the correlation between features during the training process, resulting in a large gap between the upper and lower quartiles. Compared with XGBoost and TCN models, the graph-based detection models GCN-ETA, E-minBatch GraphSAGE, and GATrust have larger differences between the upper and lower quartiles but with few outliers, which is largely due to the fact that these graph-based models consider the correlation of features. However, compared with GSA-DT model, these three graph-based detection models do not assign different weights to the features, which leads them facing feature redundancy problem and thus having relatively low detection efficiency. For the PEAN method, it has the higher accuracy, TPR and F1-measure than other six methods except for GSA-DT method.

Overall, the proposed GSA-DT model has the highest detection performance, and its detection results are concentrated in smaller boxes as well as has no outlier, which indicates that GSA-DT model has good detection stability.

G. Scalability of GSA-DT Model

To test the scalability of GSA-DT model, we use the UJS-IDS2024 dataset in the experiments, where different

sizes of network traffic are selected in the experiments. Each experiment is conducted for 30 times, and the average time cost is calculated and shown in Table V.

It can be seen from Table V that the time cost of our proposed GSA-DT model is slightly longer than that of the TCN, XGBoost and BiTCN models when processing different sizes of network traffic, which is less than three graph-based detection models of E-minBatch GraphSAGE, GATrust and GCN-ETA. For the proposed GSA-DT model, its time cost is mainly concentrated in the model training phase, which is mainly due to the fact that the training process of the model needs to transform the original network traffic into a suitable traffic topology graph to capture more correlation information, and this complex process will consume more time than traditional deep learning models (e.g., TCN). Once the GSA-DT model is trained, utilizing it to detect malicious traffic will consume much less time. Therefore, the proposed GSA-DT model has the strong scalability than other graph-based malicious traffic detection models.

H. Ablation Experiments

To verify the role of each module in the GSA-DT model, we conducted a series of ablation experiments and the experimental results are shown in Table VI. GCN indicates the original GCN model; GCN-SA indicates that self-attention mechanism is added to the original GCN model; GCN-SA-LR indicates that LeakyReLU is used in the GCN-SA model; GCN-DT indicates that decision tree is used in the GCN model; GCN-DT-LR indicates that LeakyReLU is used in the GCN-DT model; GCN-SA-DT indicates that self-attention mechanism and decision tree are added to the original GCN model.

It can be seen from the results that removing or replacing any module including LeakyReLU, decision Tree or self-attention mechanism will lead to a decrease performance. Among these modules, self-attention mechanism plays the most critical role. As can be seen from Table VI that the introduction of self-attention mechanism significantly improves the detection accuracy, which is due to that it has the ability to adaptively learn the correlation and importance between network traffic features, as well as enable the model to flexibly select features by learning the weights between different features, thereby providing the model a good generalization ability and reducing the redundancy of features. In addition, the change of activation function from the original ReLU to LeakyReLU can solve the neuron “death” problem as well as reduce the risk of gradient disappearance. Especially on the USTC-TFC2016 dataset that contains many attack types with detailed attack labels, the optimization of activation function can achieve good result. Moreover, the use of decision tree makes GSA-DT having the advantages of easy interpretation, good multi-classification capability, robustness to missing data and outliers, and efficient feature selection and training, which can improve the detection performance of GSA-DT model. The experimental results in Table VI show that the proposed GSA-DT model achieves the best detection results with better generalization ability on all four network traffic

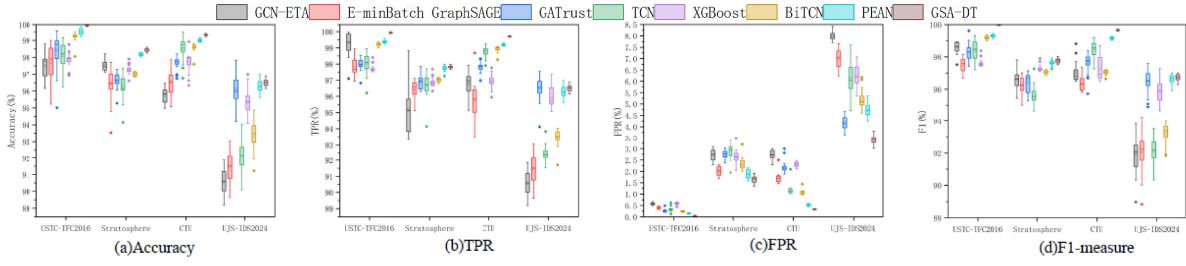


Fig. 8. The stability of eight malicious detection models.

TABLE VI
AVERAGE DETECTION ACCURACY OF DIFFERENT MODULES ON
MALICIOUS DETECTION MODELS (%)

Model	USTC-TFC2016	Stratosphere	CTU
GCN	97.96	93.65	94.81
GCN-SA	99.01	95.64	98.12
GCN-SA-LR	99.12	96.30	98.43
GCN-DT	97.50	97.51	95.52
GCN-DT-LR	99.81	98.28	99.29
GCN-SA-DT	99.31	98.30	99.25
GSA-DT	99.96	98.49	99.36

TABLE VII
TIME COST OF GSA-DT WITH GINI COEFFICIENT AND ENTROPY (SEC.)

Time cost	USTC-TFC2016	Stratosphere	CTU
GCN-DT-Gini	200.82	219.31	290.98
GCN-DT-entropy	213.60	224.74	293.21

datasets, which indicates that these improvements in GSA-DT are very effective in improving the detection of malicious traffic.

In addition, we also add the experiments to verify that the use of Gini coefficient consumes less time than entropy, and the experimental results are show in Table VII. GSA-DT-Gini means the GSA-DT model with Gini coefficient and GSA-DT-entropy means the GSA-DT model with entropy.

It can be seen from Table VII that the time cost of GSA-DT model with the use of Gini coefficient is shorter than that with the use of entropy, which is due to the fact that the computing of Gini coefficient only involves the Gini impurity of each feature, whereas the computing of entropy needs to consider the information of each feature. Experimental results demonstrate the use of Gini coefficient reducing the time cost in large-scale and complex network traffic.

I. Discussion

Extensive experimental results show that the proposed GSA-DT model can effectively solve the problems of existing malicious traffic models. For the GSA-DT model, it first performs pre-processing operations on original network traffic to obtain better network traffic and their labels, and then uses GCN to extract the topological structure of network traffic as well as capture the correlation relationships among traffic features. In the GSA-DT model, the commonly used ReLU activation function is replaced with LeakyReLU to overcome the problems of neuron “death” and gradient disappearance. In addition, GSA-DT model also introduces the self-attention mechanism in the GCN model to assign larger weights to the key features, thereby reducing the interference of redundant features. Finally, GSA-DT uses decision tree to perform the detection of malicious traffic. These improvements make

GSA-DT method show improvements in terms of accuracy, F1-measure, FPR, TPR and stability.

For the proposed GSA-DT method, it can achieve better detection accuracy due to it constructs a graph representation to capture richer interdependencies among the features of network traffic, but this process consumes much time. That is, GSA-DT method improves the detection accuracy by consuming more time, which is similar with deep learning-based malicious traffic detection methods. We can improve the time efficiency of GSA-DT using the following potential strategies: (1) Optimize the GCN using pruning and quantization techniques to remove the neurons and connections that contribute less to the detection results, thus reducing the computational cost. (2) Incorporate the operation of sparsifying the adjacency matrix before using normalization method, thus effectively reducing the memory usage and computational complexity. (3) Use the parallel computing and distributed processing to accelerate the model’s inference speed, thereby achieving the real-time processing.

There are several practical considerations that need to be considered when deploying GSA-DT model in real-world network environments: **(1) Integration with existing network infrastructure:** GSA-DT model utilizes GCN and self-attention mechanism to improve the detection accuracy, it has a relatively high demand on computational resources, thus, it is necessary to evaluate the availability of sufficient computational power in the existing network infrastructure to support real-time operation. **(2) Handling live traffic:** GSA-DT model performs malicious traffic detection through graph structure construction and feature extraction, which may lead to processing delays, therefore, the inference speed of the model needs to be optimized when deploying GSA-DT model in high-traffic environments, e.g., reducing the depth of network layers or parallelizing the processing to reduce the computational latency. **(3) Maintaining system performance under varying conditions:** The performance of GSA-DT model may vary under different network conditions, especially encountering novel cyber-attack patterns, thus, it is necessary to retrain and update periodically to make it adapt to new network environment.

V. THREATS TO VALIDITY

In Section VI, extensive experimental results demonstrate the effectiveness of GSA-DT model. However, our findings still present two major threats in the following areas.

Internal validity: We consider many parameters (including epoch number, dropout, number of hidden units, etc.) under

different situations, but not consider the parameters such as the number of network layers and filter settings, the changes of these different parameter combinations leave many possibilities. In this paper, we verify the effectiveness of GSA-DT by changing several parameters and finally select the parameter with better experimental results as the final parameter, although this manner leads to better experimental results, but it does not guarantee that the selected parameters are optimal. In addition, the used preprocessing methods include traffic analysis, traffic cleaning, information extraction and traffic labeling, but different preprocessing can lead to differences and eventually lead to changes in the experimental results, therefore, whether there is a better preprocessing method remains to be investigated. Moreover, we compare the models of GATrust [21], GCN-ETA [22], E-minBatch GraphSAG [20], TCN [36], XGBoost [37], BiTCN [17] and PEAN [19] in this paper, but we do not get the full details of the model, therefore, the experimental results may have errors.

External validity: External validity threats are mainly caused by the used network traffic datasets, where experiments are conducted using four datasets that contains various malicious traffic to validate the detection efficiency of GSA-DT, but there are more categories of malicious traffic in real life, and the used datasets are only a small fraction of the categories of intrusion traffic in real cyberspace, therefore, there is no guarantee that GSA-DT model can maintain a high detection accuracy for all malicious traffic in practical applications. In the future, we need to conduct experiments on more datasets to further validate the efficiency of GSA-DT model. In addition, the current study targets unencrypted or mixed (containing both encrypted and unencrypted traffic) malicious traffic, while the performance of GSA-DT model on encrypted network traffic is not yet known.

The attacker may attack model by analyzing the model's classification logic and generating traffic specifically designed using adversarial sample generation, which may cause the model to misreport normal traffic as malicious or misreport some malicious traffic. Therefore, the robustness of model can be improved against such adversarial attacks from the following perspectives: (1) Enhance the model by adversarial training and use adversarial generation network to construct simulated adversarial traffic as well as integrate them into training process, thereby improving the model's capability of detecting unknown adversarial traffic; (2) Use the idea of model stacking to integrate multiple models, so that even one model is successfully breached other models can still function, thus improving robustness of the model; (3) Take necessary measures (e.g., encryption, code obfuscation, etc.) from the system level to prevent the model from being easily accessed, thus making it more difficult for attackers to reverse the model; (4) Use the online learning approach to optimize and update the model instead of using a fixed offline training model, thus improving the model's adaptability to emerging attacks.

VI. CONCLUSION AND FUTURE WORK

With the popularity of Internet and the advent of digital age, the number of network attacks are increasing, which brings

great risks to individuals, organizations and society. Therefore, it is very crucial to accurately detect malicious traffic generated by network attacks to maintain network security. To achieve better detection performance, this paper proposes a malicious traffic detection model called GSA-DT. It first analyzes the IP information between different network nodes such as user communication and transaction. And then, it uses GCN to capture the topology in the network traffic from the associated features in network traffic; It also uses self-attention mechanism to assign larger weights to the key features. Finally, it uses a decision tree to detect malicious traffic. In addition, the ReLU is replaced by LeakyReLU to overcome neuron "death" problem and gradient disappearance, thereby improving the detection performance. Extensive experimental results on four network traffic datasets show that GSA-DT has better TPR, FPR, F1-measure and accuracy on most types of malicious traffic compared with seven state-of-the-arts, and it also has better stability.

The proposed GSA-DT model has the following drawbacks: (1) Its time cost in the training phase is still relatively long, which is mainly due to that long time need to be spent in processing the features of network traffic. (2) The use of self-attention mechanism may focuses excessively on specific features while neglecting others, leading to biased detection results. (3) It is unable to handle the data imbalance problem and this problem is indeed a highly publicized and important challenge in malicious traffic detection.

In the future, we would like to do the following researches to improve the detection of malicious traffic: **(1) The specific research directions:** (I) Further optimize the efficiency of GSA-DT model in handling real-time network traffic, especially in large-scale network environments; (II) Explore the model compression techniques, feature selection optimization and more efficient GCN models to improve the real-time detection performance; (III) Explore the cross-domain applications of GSA-DT model in different network environments and different types of malicious traffic detection, and study how to maintain efficient and accurate performance in IoT device networks, enterprise intranets and public cloud environments. **(2) The potential improvements in model architecture:** (I) Introduce a more complex attention mechanism such as adaptive attention models to capture more complex feature relationships; (II) Combine the GSA-DT with other detection models (e.g., deep learning models or rule-based models) to form a hybrid detection system; (III) Study how to efficiently deploy the GSA-DT models on distributed network environments or edge devices. All these improvements lead GSA-DT model achieve high detection accuracy, low-latency and high-throughput of malicious traffic detection.

REFERENCES

- [1] Z. Wang, K. W. Fok, and V. L. L. Thing, "Machine learning for encrypted malicious traffic detection: Approaches, datasets and comparative study," *Comput. Secur.*, vol. 113, Feb. 2022, Art. no. 102542.
- [2] V. C. Cunha, A. Z. Zavala, D. Magoni, P. R. M. Inácio, and M. M. Freire, "A complete review on the application of statistical methods for evaluating internet traffic usage," *IEEE Access*, vol. 10, pp. 128433–128455, 2022.

- [3] A. Blaise, M. Bouet, V. Conan, and S. Secci, "Detection of zero-day attacks: An unsupervised port-based approach," *Comput. Netw.*, vol. 180, Oct. 2020, Art. no. 107391.
- [4] H.-Y. Kwon, T. Kim, and M.-K. Lee, "Advanced intrusion detection combining signature-based and behavior-based detection methods," *Electronics*, vol. 11, no. 6, p. 867, 2022.
- [5] W. Wang, X. Du, D. Shan, R. Qin, and N. Wang, "Cloud intrusion detection method based on stacked contractive auto-encoder and support vector machine," *IEEE Trans. Cloud Comput.*, vol. 10, no. 3, pp. 1634–1646, Jul.–Sep. 2022.
- [6] A. I. Saleh, F. M. Talaat, and L. M. Labib, "A hybrid intrusion detection system (HIDS) based on prioritized k-nearest neighbors and optimized SVM classifiers," *Artif. Intell. Rev.*, vol. 51, pp. 403–443, Mar. 2019.
- [7] Y. Zhong, Z. Wang, X. Shi, J. Yang, and K. Li, "RFG-HELAD: A robust fine-grained network traffic anomaly detection model based on heterogeneous ensemble learning," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 5895–5910, 2024.
- [8] Z. Zhao et al., "ERNN: Error-resilient RNN for encrypted traffic detection towards network-induced phenomena," *IEEE Trans. Dependable Secure Comput.*, early access, Feb. 3, 2023, doi: [10.1109/TDSC.2023.3242134](https://doi.org/10.1109/TDSC.2023.3242134).
- [9] X. Han, S. Liu, J. Liu, B. Jiang, Z. Lu, and B. Liu, "ECNet: Robust malicious network traffic detection with multi-view feature and confidence mechanism," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 6871–6885, 2024.
- [10] M. Chen, Z. Wei, Z. Huang, B. Ding, and Y. Li, "Simple and deep graph convolutional networks," in *Proc. 37th Int. Conf. Mach. Learn.*, 2020, pp. 1725–1735.
- [11] J. Zhou, Z. Xu, A. M. Rush, and M. Yu, "Automating botnet detection with graph neural networks," 2020, *arXiv:2003.06344*.
- [12] L. Haghnegahdar and Y. Wang, "A whale optimization algorithm-trained artificial neural network for smart grid cyber intrusion detection," *Neural Comput. Appl.*, vol. 32, pp. 9427–9441, Jul. 2020.
- [13] X. Kan et al., "A novel IoT network intrusion detection approach based on adaptive particle swarm optimization convolutional neural network," *Inf. Sci.*, vol. 568, pp. 147–162, Aug. 2021.
- [14] Z. Chiba, N. Abghour, K. Moussaid, A. El Omri, and M. Rida, "A novel architecture combined with optimal parameters for back propagation neural networks applied to anomaly network intrusion detection," *Comput. Secur.*, vol. 75, pp. 36–58, Jun. 2018.
- [15] L. Chang and P. Branco, "Embedding residuals in graph-based solutions: The E-ResSAGE and E-ResGAT algorithms. A case study in intrusion detection," *Appl. Intell.*, vol. 54, pp. 6025–6040, May 2024.
- [16] S. Huda, S. Miah, J. Yearwood, S. Alyahya, H. Al-Dossari, and R. Doss, "A malicious threat detection model for cloud assisted Internet of Things (CoT) based industrial control system (ICS) networks using deep belief network," *J. Parallel Distrib. Comput.*, vol. 120, pp. 23–31, Oct. 2018.
- [17] J. Chen, T. Lv, S. Cai, L. Song, and S. Yin, "A novel detection model for abnormal network traffic based on bidirectional temporal convolutional network," *Inf. Softw. Technol.*, vol. 157, May 2023, Art. no. 107166.
- [18] A. Venturi, M. Ferrari, M. Marchetti, and M. Colajanni, "ARGANIDS: A novel network intrusion detection system based on adversarially regularized graph autoencoder," in *Proc. 38th ACM/SIGAPP Symp. Appl. Comput.*, 2023, pp. 1540–1548.
- [19] P. Lin, K. Ye, Y. Hu, Y. Lin, and C.-Z. Xu, "A novel multimodal deep learning framework for encrypted traffic classification," *IEEE/ACM Trans. Netw.*, vol. 31, no. 3, pp. 1369–1384, Jun. 2023.
- [20] J. Lan et al., "E-minBatch GraphSAGE: An industrial internet attack detection model," *Secur. Commun. Netw.*, vol. 2022, Jul. 2022, Art. no. 5363764.
- [21] N. Jiang, J. Wen, J. Li, X. Liu, and D. Jin, "GATrust: A multi-aspect graph attention network model for trust assessment in OSNs," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 6, pp. 5865–5878, Jun. 2023.
- [22] J. Zheng, Z. Zeng, and T. Feng, "GCN-ETA: High-efficiency encrypted malicious traffic detection," *Secur. Commun. Netw.*, vol. 2022, Jan. 2022, Art. no. 4274139.
- [23] J. Xiao, L. Yang, and S. Wang, "Graph isomorphism network for materials property prediction along with explainability analysis," *Comput. Mater. Sci.*, vol. 233, Jan. 2024, Art. no. 112619.
- [24] L. Zhao et al., "T-GCN: A temporal graph convolutional network for traffic prediction," *IEEE Trans. Intell. Transp. Syst.*, vol. 21, no. 9, pp. 3848–3858, Sep. 2020.
- [25] C. Fu, Q. Li, M. Shen, and K. Xu, "Frequency domain feature based robust malicious traffic detection," *IEEE/ACM Trans. Netw.*, vol. 31, no. 1, pp. 452–467, Feb. 2023.
- [26] T.-Y. Kim and S.-B. Cho, "Web traffic anomaly detection using C-LSTM neural networks," *Expert Syst. Appl.*, vol. 106, pp. 66–76, Sep. 2018.
- [27] Y. Qu, R. K. Baghbaderani, H. Qi, and C. Kwan, "Unsupervised pansharpening based on self-attention mechanism," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 4, pp. 3192–3208, Apr. 2021.
- [28] Y. Ding, M. Jia, Q. Miao, and Y. Cao, "A novel time–frequency transformer based on self-attention mechanism and its application in fault diagnosis of rolling bearings," *Mech. Syst. Signal Process.*, vol. 168, Apr. 2022, Art. no. 108616.
- [29] M.-H. Guo, Z.-N. Liu, T.-J. Mu, and S.-M. Hu, "Beyond self-attention: External attention using two linear layers for visual tasks," 2021, *arXiv:2105.02358*.
- [30] V. G. Costa and C. E. Pedreira, "Recent advances in decision trees: An updated survey," *Artif. Intell. Rev.*, vol. 56, no. 5, pp. 4765–4800, 2023.
- [31] W. Wang, M. Zhu, X. Zeng, X. Ye, and Y. Sheng, "Malware traffic classification using convolutional neural network for representation learning," in *Proc. Int. Conf. Inf. Netw. (ICOIN)*, 2017, pp. 712–717.
- [32] 2015, "Stratosphere laboratory Datasets," Dataset, Stratosphere, Mar. 13, 2020. [Online]. Available: <https://www.stratosphereips.org/datasets-overview>
- [33] O. Roques, S. Maffei, and M. Cova, "Detecting malware in TLS traffic," in *Proc. IEEE Conf. Local Comput. Netw. 30th Anniv. (LCN)*, 2019, pp. 1–77.
- [34] K. Sood, M. R. Nosouhi, D. D. N. Nguyen, F. Jiang, M. Chowdhury, and R. Doss, "Intrusion detection scheme with dimensionality reduction in next generation networks," *IEEE Trans. Inf. Forensics Security*, vol. 18, pp. 965–979, 2023.
- [35] H. Xu, S. Han, X. Li, and Z. Han, "Anomaly traffic detection based on communication-efficient federated learning in space-air-ground integration network," *IEEE Trans. Wireless Commun.*, vol. 22, no. 12, pp. 9346–9360, Dec. 2023.
- [36] J. Chen, S. Yin, S. Cai, C. Zhang, Y. Yin, and L. Zhou, "An efficient network intrusion detection model based on temporal convolutional networks," in *Proc. IEEE 21st Int. Conf. Softw. Qual., Rel. Security (QRS)*, 2021, pp. 768–775.
- [37] Y. Song, H. Li, P. Xu, and D. Liu, "A method of intrusion detection based on WOA-XGBoost algorithm," *Discr. Dyn. Nat. Soc.*, vol. 2022, no. 1, 2022, Art. no. 5245622.



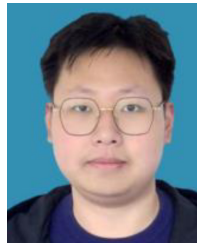
Saihua Cai (Member, IEEE) received the Ph.D. degree from the College of Information and Electrical Engineering, China Agricultural University, Beijing, China, in 2020. He is currently an Associate Professor with the School of Computer Science and Communication Engineering, Jiangsu University, Zhenjiang, China. He has published more than 90 papers in some famous journals or conferences, including *ACM Transactions on Privacy and Security*, the *IEEE TRANSACTIONS ON RELIABILITY*, *ASE*, and *ISSRE*. His major research interests include malicious traffic detection, outlier detection, and software testing. He is a member of ACM, and a Senior Member of the China Computer Federation.



Han Tang received the B.E. degree in software engineering from Pujiang College, Nanjing, China, in 2022. She is currently pursuing the master's degree in computer technology with Jiangsu University, Zhenjiang, China. Her research interest includes malicious traffic detection.



Jinfu Chen (Member, IEEE) received the Ph.D. degree in computer science and technology from the Huazhong University of Science and Technology, Wuhan, China, in 2009. He is currently a Full Professor with the School of Computer Science and Communication Engineering, Jiangsu University, Zhenjiang, China. He has published more than 100 papers in some famous journals or conferences, including the IEEE TRANSACTIONS ON RELIABILITY, *Information Sciences*, *Information and Software Technology*, and *Software: Practice and Experience*. His major research interests include software testing, software security, and trusted software. He is a member of ACM, and the China Computer Federation.



Wenjun Zhao received the B.E. degree in software engineering from Yancheng Teachers College in 2022. He is currently pursuing the master's degree in computer technology with Jiangsu University, Zhenjiang, China. His research interests include malicious traffic detection and data augmentation.



Tianxiang Lv received the M.D. degree in computer science and technology from Jiangsu University, Zhenjiang, China, in 2023. He has published two papers in *Information and Software Technology* and *QRS*. His major research interest includes malicious traffic detection.



Chunlei Huang received the B.E. degree in automation from the Nanjing Institute of Technology, Nanjing, China, in 2012. He is currently pursuing the master's degree in software engineering with Jiangsu University, Zhenjiang, China. He is currently working with Zhenjiang Maternal and Child Health Hospital, Zhenjiang. His research interests include malicious traffic detection and healthcare insurance fraud detection.